

Recruitment Management System — Deliverable Report

Candidate Pipeline Tracker & Natural Language Search Platform

GitHub Repository Link:

<https://github.com/ayushprajapati3002/RecruitmentSoftware>

(Contains the full project source code, README setup guide, and the `AI_CHAT_LOGS/` directory)

1. Summary of Architecture

The system is built as a modular client-server web application designed for high responsiveness, pipeline traceability, and fast conversational recruiter querying:

- **Frontend (React 19 + TypeScript + Vite):** Provides an interactive candidate recruitment pipeline (Stages: *Applied* → *Screening* → *Interview* → *Offer* → *Hired*). It includes a conversational search bar that displays the interpreted search intent and feedback, plus a candidate detail dossier drawer. Styled using clean, responsive CSS tailored for all screen sizes (mobile, foldables, tablets, and desktop).
- **Backend (Express 5 + Node.js):** RESTful API server with dedicated controllers for candidate CRUD, stage movement (advance/reject/archive), and search endpoints. Uses `Multer` to safely handle resume file uploads.
- **Database (PostgreSQL + Prisma ORM):** Stores candidates with normalized search fields (such as digits-only phone numbers) and an append-only, immutable `candidate_history` audit table that logs every stage transition with exact timestamps and actions.
- **Natural Language Processing (Groq + Llama-3.3-70B):** Recruiter queries like *"candidates in screening with 3+ years experience"* are translated in under 250ms into a deterministic JSON filter structure, which the backend safely executes as parameterized SQL queries.

2. How to Run the Application

1. **Prerequisites:** Node.js (v18+ or v24+), a PostgreSQL database instance, and a free Groq API key.

2. Backend Setup:

```
cd backend
npm install
# Configure backend/.env with DATABASE_URL and GROQ_API_KEY
npx prisma db push
npm run dev      # Runs on http://localhost:5000
```

3. Frontend Setup:

```
cd frontend
npm install
npm run dev      # Runs on http://localhost:5173
```

3. Key Decisions Made and Why

- **Groq Llama-3.3-70B for Natural Language Search:** We chose high-speed cloud LLM inference over brittle regex parsing or local heavy Python NLP models. It extracts complex recruiter intents accurately with sub-250ms latency and outputs strict JSON without runtime overhead.
- **Immutable `CandidateHistory` Table:** Instead of overwriting a single status column, every candidate movement (advance, reject, create) creates an append-only audit record with `onDelete: NoAction`. This guarantees a verifiable compliance history.
- **Soft Deletes (`isArchived`):** Candidates are never permanently purged from the database. Archiving retains full candidate profile history and past hiring evaluations for future openings.
- **Vanilla CSS Design System:** Used custom CSS grid and flexbox without large UI libraries (like Tailwind or Bootstrap) to ensure total layout control across all device viewports, especially narrow phones and foldables.
- **Local Disk Uploads via Multer:** Resumes are stored on disk with unique timestamp hashes rather than stored as large base64 strings in the database, avoiding database bloat.

4. AI Collaboration: Where I Disagreed with the AI

The Incident: Rule-Based Offline Regex Parsing vs. Pure LLM Intent Extraction

During the implementation of the natural language search feature, the AI proposed a two-step parsing strategy starting with *"Step 1: Offline Rule-Based NLP"*. The AI wrote regular expression heuristics (e.g., `/(applied|screening|interview|offer|hired)/i`, `/(\d+)\+?\s*years?/i`) to intercept queries before passing them to the model.

My Directive: *"Don't want this Step 1: Offline Rule-Based NLP. Remove the changes of step 1 only."*

Why I Disagreed:

- **False Positives:** Natural conversational recruiter queries (such as *"candidates who passed screening for backend developer"*) easily trick simple regex patterns, confusing notes or job descriptions with pipeline stages.
- **Duplicate Maintenance:** Having both a rigid regex ruleset and an LLM pipeline introduces duplicate parsing logic and edge cases.
- **Speed:** Because Groq's Llama-3 model responds in under 250ms, a fragile offline regex fallback was completely unnecessary. I insisted on a single, clean pipeline where the LLM produces a strict JSON schema and the backend constructs parameterized SQL queries.

5. What I'd Do With More Time

- **Semantic Vector Search (pgvector):** Add vector embeddings over candidate resumes and interview notes to allow conceptual queries like *"find candidates with leadership experience in cloud migrations"*.
- **Automated Resume OCR & Parsing:** Automatically parse uploaded PDF/Word resumes to auto-fill skills, work history, and contact information.
- **Interview Scheduling Integration:** Integrate Google Calendar / Outlook APIs to let recruiters send interview invitations and view open slots directly from the candidate card.
- **Real-time Multi-Recruiter Sync:** Use WebSockets or Server-Sent Events (SSE) so stage changes and notes update live for all recruiters viewing the board.
- **Role-Based Access Control (RBAC):** Add distinct permissions for Admins, Hiring Managers, and Interviewers.